

ELO312 Laboratorio de Estructura de Computadores

Actividad autoguiada: Programación en C

Rodrigo Jiménez, Mauricio Solís, Alejandro Weinstein
20260805

Esta actividad autoguiada tiene como objetivo que Ud. se familiarice con los principios básicos del lenguaje de programación C. Para ello, realice individualmente las actividades descritas a continuación, procurando escribir el código de cada actividad para sacarle el máximo provecho a esta experiencia (no haga copiar y pegar). Esta actividad debe realizarse fuera del horario del laboratorio, antes de la primera sesión guiada. Si bien la realización de estas actividades no será verificada, el manejo de estos conceptos es necesario para llevar a cabo la primera sesión guiada.

Ejecute los códigos en cualquier computador que cuente con un entorno de desarrollo para C. En Windows puede usar, por ejemplo, DevCpp¹. Otro entorno que les puede servir y que es multiplataforma (Windows, Linux, Mac) es CodeBlocks². También puede usar el compilador en línea *One Compiler*³.

1. Operaciones aritméticas

Considere el siguiente programa.

```
1 #include <stdio.h>
2
3 int a;
4 int b = 10;
5
6 int main()
7 {
8     int b = 62;
9     a = b;
10
11     int res1 = a * b;
12     int res2 = a - 72;
13     unsigned int res3 = a - 72;
14     unsigned short res4 = a - 72;
15     int res5 = res3 + 72;
16     int res6 = res4 + 72;
17     int res7 = 5 / 2;
18     float res8 = 5 / 2;
19     float res9 = 5 / 2.0;
20     int res10 = 5 % 2;
21     char res11 = b;
22     char res12 = 0xFF + 1;
23
24     // Ver https://en.cppreference.com/w/c/io/fprintf
25     // para los detalles sobre el "conversion specifier"
26
```

¹Instalador disponible en https://github.com/Embarcadero/Dev-Cpp/releases/download/v6.3/Embarcadero_Dev-Cpp_6.3-TDM-GCC_9.2_Setup.exe

²<https://www.codeblocks.org/>

³<https://onecompiler.com/c>

```

27     printf("Actividad 1\n");
28     printf("res1: %d\n", res1);
29     printf("res1: %x\n", res1);
30     printf("res2: %d\n", res2);
31     printf("res3: %d\n", res3);
32     printf("res3: %u\n", res3);
33     printf("res3: %X\n", res3);
34     printf("res4: %d\n", res4);
35     printf("res5: %d\n", res5);
36     printf("res6: %d\n", res6);
37     printf("res7: %d\n", res7);
38     printf("res8: %f\n", res8);
39     printf("res9: %f\n", res9);
40     printf("res10: %d\n", res10);
41     printf("res11: %c\n", res11);
42     printf("res12: %d\n", res12);
43
44     return 0;
45 }

```

- Antes de ejecutar el código, ¿cuál espera que sea el resultado?
- Compare su respuesta a la pregunta anterior con el resultado obtenido al ejecutar el programa.
- Explique por qué **res4** vale 65526 y no -10. Luego modifique el programa para que **res4** supere el valor máximo representable y explique el resultado.

2. Bucles

2.1. Ciclo for

Considere el siguiente programa:

```

1     #include <stdio.h>
2     int main()
3     {
4         int a = 0;
5         for(int i = 0; i <= 5; i = i + 1)
6         {
7             printf("a = %d \n", a);
8             a = a + i;
9         }
10        return 0;
11    }

```

a: 0	i: 0	Para ejecutar el cuerpo tiene que cumplirse la condición, al final de ciclo actualiza la variable i = 5 -> a:10 -> i = 6 -> return
a: 0	i: 1	
a: 1	i: 2	
a: 3	i: 3	
a: 6	i: 4	
a: 10	i: 5	
a: 15	i: 6	

- ¿Qué indica cada una de las tres cláusulas de la sentencia **for**?
- Antes de ejecutar el programa, ¿cuál espera que sea el resultado?
- Compare su respuesta a la pregunta anterior con el resultado obtenido al ejecutar el programa.

2.2. Ciclo while

Considere el siguiente programa:

```
1  #include <stdio.h>
2  int main()
3  {
4      int b = 0, c = 0;
5
6      while(b < 5)
7      {
8          c = c + 2;
9          b++;
10         printf("b = %d ; c = %d\n", b, c);
11     }
12     return 0;
13 }
```

b = 1 ; c = 2
b = 2 ; c = 4
b = 3 ; c = 6
b = 4 ; c = 8
b = 5 ; c = 10

Antes de ejecutar el programa, responda las siguientes preguntas:

- ¿Cuántas veces se repite el ciclo? 5
- ¿Cuánto valdría c si el valor inicial de b fuese 5? 0
- ¿Qué hace la sentencia b++? usa el valor antiguo de "b" y luego lo incrementa

Ejecute su programa para verificar sus respuestas a las preguntas anteriores.

2.3. Ciclo do-while

Considere el siguiente programa:

```
1  #include <stdio.h>
2  int main()
3  {
4      int b = 0, c = 0;
5
6      do
7      {
8          c = c + 2;
9          b++;
10         printf("b = %d ; c = %d\n", b, c);
11     }
12     while(b < 5);
13     return 0;
14 }
```

Antes de ejecutar el programa, responda las siguientes preguntas:

- ¿Cuántas veces se repite el ciclo? 5
- ¿Cuánto valdría c si el valor inicial de b fuese 5? 2
- ¿Cómo se diferencia de la sentencia while del punto anterior? while pregunta la condición y luego ejecuta do-while ejecuta y luego pregunta

Ejecute su programa para verificar sus respuestas a las preguntas anteriores.

3. Control de flujo

Considere los siguientes programas:

3.1. if - else

```
1  #include <stdio.h>
2  int main()
3  {
4      int a = 0;
5      for(int i = 0; i < 5; i = i + 1)
6      {
7          a = a + i;
8          if((i % 2) == 0)
9          {
10             a += 1;
11         }
12         else
13         {
14             a = a + 3;
15         }
16         printf("i = %d ; a = %d\n", i, a);
17     }
18     return 0;
19 }
```

- Antes de ejecutar el programa, ¿cuál espera que sea el resultado?
- Compare su respuesta a la pregunta anterior con el resultado obtenido al ejecutar el programa.
- ¿Es necesario utilizar la sentencia `else` después de un `if`? ¿Por qué?

Ejecute su programa para verificar sus respuestas a las preguntas anteriores.

3.2. switch - case

```
1  #include <stdio.h>
2  int main()
3  {
4      int b = 0, c = 0;
5      while(b < 5)
6      {
7          switch(b)
8          {
9              case 0:
10                 c = 0;
11                 break;
12              case 1:
13                 c++;
14              case 2:
15                 c--;
16                 break;
17              default:
18                 c = b;
19             }
20         }
21     }
```

```

19         break;
20     }
21     c = c+2;
22     b++;
23     printf("b = %d ; c = %d\n", b, c);
24 }
25 return 0;
26 }

```

- Antes de ejecutar el programa, ¿cuál espera que sea el resultado?
- Compare su respuesta a la pregunta anterior con el resultado obtenido al ejecutar el programa.
- ¿Qué hace la instrucción `break`? ¿Qué pasaría si no se incluye? sigue a comprobar la siguiente condición
- ¿Es necesario utilizar la sentencia `default` en un `switch - case`? ¿Por qué?

Ejecute su programa para verificar sus respuestas a las preguntas anteriores.

3.3. Operador ternario

```

1  #include <stdio.h>
2  int main()
3  {
4      int b = 0, c = 0;
5      do
6      {
7          c = (b == 0)? 0 : (b == 1)? c+1 : b;
8          b++;
9          printf("b = %d ; c = %d\n", b, c);
10     }
11     while(b < 5);
12     return 0;
13 }

```

- Antes de ejecutar el programa, ¿cuál espera que sea el resultado de ejecutarlo?
- Compare su respuesta a la pregunta anterior con el resultado obtenido al ejecutar el programa.
- ¿Se puede obviar la última parte (`: b`)?

Ejecute su programa para verificar sus respuestas a las preguntas anteriores.

4. Operaciones *bitwise*

Cree un nuevo programa que incluya las siguientes variables globales:

```

1  unsigned int op1 = 0x00FF, op2 = 0x0F0A, op3 = 4;
2  unsigned int res1, res2, res3, res4, res5,
3              res6, res7, res8, res9, res10;

```

Luego, agregue las siguientes líneas de código dentro de la función `main`:

```

1  res1 = op1 && op2;
2  res2 = op1 & op2;
3  res3 = op1 || op2;
4  res4 = op1 | op2;
5  res5 = op1 ^ op2;
6  res6 = !op1;
7  res7 = ~op2;
8  res8 = op3 << 2;
9  res9 = op3 >> 2;
10 res10 = op2 << 4;

```

- Clasifique los operadores en lógicos y bit a bit (*bitwise*). ¿Por qué **res1** vale 1 y **res2** vale 0x0A si ambos operan sobre los mismos datos?
- Agregue el código necesario para imprimir cada una de las variables del programa.
- Antes de ejecutar el programa, ¿cuál espera que sea el resultado de ejecutarlo?
- Compare su respuesta a la pregunta anterior con el resultado obtenido al ejecutar el programa.
- ¿Qué significa desplazar una variable 1 posición a la derecha? ¿Y a la izquierda?
- ¿Para qué utilizaría cada una?

multiplicar o dividir por 2

& para reset variables y mantener otras
| para setear variables y mantener otras
~ para negar pines negados

Ejecute su programa para verificar sus respuestas a las preguntas anteriores
para ver si 2 variables son iguales o distintas
<< para crear numeros potencias de 2

5. Máscaras de bits

Las operaciones *bitwise* se utilizan principalmente para modificar *algunos* bits de una variable sin alterar el resto. Esta es la forma en que se configuran los periféricos de un microcontrolador, en la que cada bit de un registro cumple una función distinta. Considere el siguiente programa:

```

1  #include <stdio.h>
2
3  unsigned int reg = 0x0F0A;
4
5  int main()
6  {
7      unsigned int n = 5;
8
9      printf("inicial : 0x%08X\n", reg);
10
11     reg |= (1u << n);                /* (1) */
12     printf("(1)      : 0x%08X\n", reg);
13
14     reg &= ~(1u << n);                /* (2) */
15     printf("(2)      : 0x%08X\n", reg);
16
17     reg ^= (1u << n);                /* (3) */
18     printf("(3)      : 0x%08X\n", reg);
19
20     printf("bit %u    : %u\n", n, (reg >> n) & 1u); /* (4) */
21
22     return 0;
23 }

```

- a. Escriba en binario el valor inicial de `reg` y el valor de la máscara (`1u << n`). ¿Qué operación realiza cada una de las líneas marcadas con (1), (2), (3) y (4)?
- b. ¿Por qué las líneas (1), (2) y (3) no modifican ningún bit distinto del bit `n`? Justifique a partir de la tabla de verdad de cada operador.
- c. Antes de ejecutar el programa, ¿cuál espera que sea el resultado de ejecutarlo?
- d. Compare su respuesta a la pregunta anterior con el resultado obtenido al ejecutar el programa.
- e. ¿Qué diferencia hay entre las instrucciones `reg = 0x20;` y `reg |= 0x20;`? ¿En cuál de las dos se pierde el valor previo de los demás bits?
- f. Escriba las instrucciones que, sobre una variable `reg` de 32 bits, realicen cada una de las siguientes acciones sin alterar los bits no involucrados:
 - I. poner en 1 el bit 5;
 - II. poner en 0 el bit 5;
 - III. invertir el bit 5;
 - IV. poner en 1 simultáneamente los bits 3 y 7;
 - V. poner en 0 el campo formado por los bits 8 a 11;
 - VI. escribir el valor `0x9` en el campo formado por los bits 8 a 11 (note que primero debe borrar el campo).
- g. Reescriba las tres primeras instrucciones del punto anterior utilizando las siguientes macros. ¿Qué ventaja tiene esta forma de escribirlas?

```

1 #define BIT_SET(reg, n)    ((reg) |=  (1u << (n)))
2 #define BIT_CLEAR(reg, n) ((reg) &= ~(1u << (n)))
3 #define BIT_TOGGLE(reg, n) ((reg) ^=  (1u << (n)))
4 #define BIT_READ(reg, n)  (((reg) >> (n)) & 1u)

```

Agiliza el proceso de lectura y escritura de bits en memoria

"u" viene de unsigned, cuando se trabajan con muchos bits, el ultimo corresponde al signo, por lo que es mejor asegurar que se puede hacer el shift en todo el recorrido. También existe "UL" o "ULL"

- h. ¿Por qué la máscara se escribe como `1u` y no como `1`?

Ejecute su programa para verificar sus respuestas a las preguntas anteriores.